MASTER PROJECT PROMPT

Autonomous Agentic AI Marketing & Revenue Operations Platform

You are a **Senior Agentic AI Architect, AI Product Engineer, Marketing Strategist, Growth Architect, Revenue Operations Expert, and SaaS Platform Engineer**.

Your task is to design and build an **enterprise-grade autonomous Agentic AI Marketing & Revenue Operations Platform** from end to end.

The system must function like an **AI-powered marketing department** capable of researching markets, understanding customers, creating strategies, generating content, planning campaigns, managing leads, supporting sales, analyzing performance, and continuously optimizing marketing activities.

The platform should not behave like a simple chatbot.

It must operate as a **multi-agent AI system with specialized agents, shared memory, tools, workflows, RAG, analytics, planning, evaluation, human approval, and autonomous decision-making**.

---

1. PRODUCT VISION

Build a platform that allows a company to provide:

- Company information
- Products/services
- Target market
- Customer segments
- Marketing goals
- Existing marketing assets
- Competitor information
- Historical campaign data
- CRM data
- Website information
- Social media information
- Analytics data

The AI platform then creates and operates a complete marketing engine.

The platform should answer:

> "What should the company do next to generate more qualified leads and revenue?"

It should continuously:

1. Research
2. Analyze
3. Plan
4. Execute
5. Measure
6. Learn
7. Optimize
8. Report

---

2. PRIMARY BUSINESS OBJECTIVES

The system must optimize for:

Growth

- Qualified lead generation
- Pipeline generation
- Revenue growth
- Brand awareness
- Customer acquisition
- Customer retention

Marketing efficiency

- CAC reduction
- CPL reduction
- Marketing ROI improvement
- Conversion-rate improvement
- Budget optimization

Sales acceleration

- Better lead qualification
- Better prospect intelligence
- Personalized outreach
- Sales recommendations
- Competitive intelligence

Customer success

- Retention
- Churn reduction
- Customer engagement
- NPS improvement

• Upsell/cross-sell opportunities

---

3. ORGANIZATION CONFIGURATION

Create an organization onboarding system.

The organization should configure:

```
Organization
├── Company Information
├── Industry
├── Products
├── Services
├── Geography
├── Target Markets
├── ICP
├── Buyer Personas
├── Business Goals
├── Revenue Goals
├── Marketing Budget
├── Sales Cycle
├── Brand Guidelines
├── Competitors
├── Existing Marketing Channels
├── CRM
├── Website
├── Social Accounts
└── Analytics Platforms
```

Example:

```
{
  "industry": "B2B SaaS",
  "products": [
    "AI Customer Support Platform",
    "AI Sales Agent"
  ],
  "target_market": [
    "India",
    "United States"
  ],
  "customer_type": "SMB and Mid-Market",
  "monthly_marketing_budget": 500000,
```

```
  "monthly_lead_target": 500,
  "revenue_target": 10000000
}
```

---

4. SYSTEM ARCHITECTURE

Use a modular architecture.

Recommended stack:

Frontend

```
Next.js
React
TypeScript
Tailwind CSS
Shadcn/UI
Recharts
```

Backend

```
Python
FastAPI
Pydantic
SQLAlchemy
PostgreSQL
Redis
Celery / background workers
```

AI

```
LLM Provider abstraction
OpenAI-compatible APIs
Anthropic-compatible APIs
Local models where appropriate
LangGraph
LangChain
MCP
Structured Outputs
Tool Calling
```

Vector Database

Support:

```
ChromaDB
pgvector
Pinecone
Weaviate
```

Default:

```
PostgreSQL + pgvector
```

Storage

```
S3-compatible object storage
```

Authentication

```
JWT
OAuth
RBAC
Organization-level isolation
```

Deployment

```
Docker
Docker Compose
CI/CD
Cloud deployment
```

---

5. MULTI-AGENT ARCHITECTURE

Create the following specialized agents.

Agent 1 — Market Research Agent

Responsibilities:

- Industry research
- Market trends
- Market size research
- Customer trends
- Emerging opportunities
- Industry news
- Technology trends

Output:

```
{
  "market_summary": "",
  "trends": [],
  "opportunities": [],
  "threats": [],
  "recommended_actions": []
}
```

---

6. COMPETITIVE INTELLIGENCE AGENT

Analyze competitors.

Monitor:

- Competitor websites
- Product launches
- Pricing
- Positioning
- Content
- SEO
- Social media
- Reviews
- Customer sentiment
- Advertising messaging

Generate:

```
Competitor Profile
Competitive Matrix
```

```
SWOT
Messaging Comparison
Pricing Comparison
Content Gap
SEO Gap
Opportunity Map
```

Never invent competitor information.

Clearly distinguish:

```
Verified Data
Inferred Data
AI Hypothesis
```

---

7. CUSTOMER PERSONA AGENT

Build:

ICP

- Industry
- Company size
- Revenue
- Geography
- Technology stack
- Buying capacity
- Business problem

Buyer Persona

- Role
- Goals
- Pain points
- Buying triggers
- Objections
- Decision criteria
- Preferred channels

Customer Journey

```
Awareness
↓
```

```
Problem Recognition
↓
Research
↓
Evaluation
↓
Decision
↓
Purchase
↓
Onboarding
↓
Retention
↓
Expansion
```

---

8. BRAND STRATEGY AGENT

Create:

- Brand positioning
- Value proposition
- Messaging framework
- Brand voice
- Brand personality
- Differentiation
- Elevator pitch
- Website messaging
- Sales messaging

Maintain a centralized:

```
Brand Knowledge Base
```

All content agents must retrieve brand guidelines before generating public-facing content.

---

9. CONTENT STRATEGY AGENT

Generate:

- Blogs

- LinkedIn posts
- Instagram content
- YouTube scripts
- Whitepapers
- Case studies
- Ebooks
- Landing pages
- Newsletters
- Webinars
- Email sequences
- Ad copy

Every content asset must include:

```
{
  "objective": "",
  "audience": "",
  "funnel_stage": "",
  "content_type": "",
  "topic": "",
  "cta": "",
  "kpi": [],
  "expected_business_impact": ""
}
```

---

10. SEO AGENT

Perform:

- Keyword research
- Search intent analysis
- Competitor keyword analysis
- Content gap analysis
- On-page SEO
- Technical SEO recommendations
- Internal linking
- Topic clustering

Create:

```
Keyword Database
Topic Clusters
Pillar Pages
```

```
Supporting Articles
Content Calendar
SEO Roadmap
```

Score keywords using:

```
Business Value
Search Intent
Competition
Search Volume
Conversion Potential
```

---

11. SOCIAL MEDIA AGENT

Support:

```
LinkedIn
Instagram
Facebook
YouTube
X
```

Generate:

- Content calendars
- Posts
- Carousels
- Short-video scripts
- Thought leadership
- Industry insights
- Engagement responses

Monitor:

```
Reach
Impressions
Engagement
CTR
Shares
Comments
Followers
```

```
Leads
Conversions
```

---

12. EMAIL MARKETING AGENT

Generate:

Lifecycle campaigns

```
Welcome
Lead Nurturing
Product Education
Demo Follow-up
Sales Follow-up
Re-engagement
Retention
Upsell
Cross-sell
Win-back
```

Use personalization.

Example:

```
Company
Industry
Job Role
Pain Point
Previous Interaction
Product Interest
Buying Intent
```

---

13. CAMPAIGN MANAGEMENT AGENT

Create campaigns across:

```
Google Ads
LinkedIn
Meta
```

```
Email
Organic Search
Events
Webinars
Partnerships
Referrals
Content
```

Each campaign must contain:

```
{
  "campaign_name": "",
  "objective": "",
  "audience": "",
  "budget": 0,
  "channels": [],
  "message": "",
  "creative_requirements": [],
  "landing_page": "",
  "kpis": [],
  "target_cpl": 0,
  "target_cac": 0,
  "expected_pipeline": 0
}
```

---

14. LEAD GENERATION AGENT

Create lead-generation workflows.

Lead sources:

```
Website
Forms
Content Downloads
Events
Webinars
Organic Search
Paid Ads
Referrals
Partnerships
CRM
```

Create lead scoring.

Example:

```
Lead Score =
Company Fit
+
Role Fit
+
Engagement
+
Intent
+
Behavior
```

Classify:

```
Cold
Warm
Marketing Qualified
Sales Qualified
High Intent
Customer
```

---

15. SALES ENABLEMENT AGENT

For every high-value prospect generate:

```
Account Summary
Company Research
Decision Maker
Pain Points
Likely Needs
Potential Objections
Competitors
Personalized Outreach
Follow-up Strategy
Recommended Product
```

Generate:

- Sales emails
- LinkedIn messages
- Call preparation
- Meeting briefs
- Battle cards
- Proposal assistance
- Objection handling

---

16. ANALYTICS AGENT

Create a centralized marketing analytics engine.

Track:

Awareness

```
Reach
Impressions
Website Traffic
Brand Searches
```

Engagement

```
CTR
Engagement Rate
Time on Site
Content Engagement
```

Leads

```
Leads
MQL
SQL
Conversion Rate
Lead Quality
```

Revenue

```
Pipeline
Pipeline Velocity
Revenue Influenced
Revenue Generated
```

Efficiency

```
CAC
CPL
ROAS
ROI
```

Customer

```
Retention
Churn
LTV
NPS
Expansion Revenue
```

---

17. MARKETING AUTOMATION AGENT

Create event-driven workflows.

Example:

```
User visits pricing page
        ↓
Track event
        ↓
Increase intent score
        ↓
Update CRM
        ↓
Lead scoring agent
        ↓
If high intent
        ↓
Create sales alert
```

```
        ↓
Generate personalized outreach
```

Another example:

```
Customer inactive for 30 days
        ↓
Retention Agent
        ↓
Analyze customer behavior
        ↓
Identify churn risk
        ↓
Generate intervention
        ↓
Notify Customer Success
```

---

18. CUSTOMER RETENTION AGENT

Monitor:

- Product usage
- Engagement
- Customer support activity
- Purchase history
- Satisfaction
- NPS
- Inactivity

Calculate:

```
Churn Risk Score
```

Generate:

```
Retention Recommendation
Upsell Opportunity
Cross-sell Opportunity
Customer Health Score
```

---

19. CUSTOMER FEEDBACK AGENT

Collect:

```
Reviews
Support tickets
Surveys
NPS
Customer interviews
Social comments
Feedback forms
```

Perform:

```
Sentiment Analysis
Topic Extraction
Pain Point Detection
Feature Request Detection
Complaint Detection
```

Create:

```
Voice of Customer Report
```

---

20. AGENT ORCHESTRATOR

Create a central:

```
Marketing Orchestrator Agent
```

Responsibilities:

- Understand user goals
- Break objectives into tasks
- Select appropriate agents
- Execute workflows
- Maintain state
- Validate outputs
- Resolve conflicts

- Trigger human approval
- Track results
- Re-plan when results are poor

Example:

```
User:
"We need 500 qualified leads in 60 days."

Orchestrator:

1. Market Research Agent
2. Customer Persona Agent
3. Competitive Agent
4. Lead Generation Agent
5. Content Agent
6. SEO Agent
7. Campaign Agent
8. Email Agent
9. Analytics Agent

Then combine outputs into:

60-Day Growth Plan
```

---

21. LANGGRAPH WORKFLOW

Use LangGraph to implement stateful agent workflows.

Example state:

```python
class MarketingState(TypedDict):

    organization_id: str

    objective: str

    market_research: dict

    competitors: list

    personas: list
```

```
    campaigns: list

    content: list

    leads: list

    analytics: dict

    recommendations: list

    approvals: list

    execution_status: str
```

Workflow:

```
START
 ↓
Understand Objective
 ↓
Research
 ↓
Persona Analysis
 ↓
Strategy
 ↓
Campaign Planning
 ↓
Content Creation
 ↓
Approval
 ↓
Execution
 ↓
Analytics
 ↓
Evaluation
 ↓
Optimization
 ↓
REPORT
 ↓
END
```

---

22. RAG SYSTEM

Build an enterprise marketing knowledge base.

Documents may include:

```
Company Documents
Product Documentation
Brand Guidelines
Pricing
Case Studies
Sales Decks
Customer Reviews
Competitor Reports
Marketing Reports
Previous Campaigns
FAQs
Website Content
```

Pipeline:

```
Document
 ↓
Parser
 ↓
Chunking
 ↓
Metadata
 ↓
Embedding
 ↓
Vector Database
```

Use metadata:

```
{
  "organization_id": "",
  "document_type": "",
  "department": "",
  "product": "",
  "created_at": "",
```

```
  "source": ""
}
```

Implement:

```
Hybrid Search
Semantic Search
Keyword Search
Metadata Filtering
Reranking
```

---

23. MEMORY SYSTEM

Implement three memory layers.

Short-Term Memory

Current task/workflow state.

Long-Term Memory

Persistent organization knowledge:

```
Preferences
Past Campaigns
Successful Messaging
Customer Insights
Brand Rules
Historical Decisions
```

Episodic Memory

Store:

```
What action was taken
Why it was taken
What happened
What worked
What failed
```

Use this information for future optimization.

24. MCP TOOL ARCHITECTURE

Build an MCP-compatible tool layer.

Example tools:

```
search_web
search_company
search_competitor
get_website
analyze_website
get_google_analytics
get_crm_leads
create_lead
update_lead
send_email
create_campaign
get_campaign_metrics
create_social_post
schedule_social_post
generate_content
search_knowledge_base
get_customer
calculate_lead_score
generate_report
```

Agents must call tools rather than hallucinate external information.

---

25. TOOL PERMISSION SYSTEM

Every tool must define:

```
{
  "tool": "send_email",
  "risk_level": "high",
  "requires_approval": true
}
```

Risk levels:

```
LOW
MEDIUM
HIGH
CRITICAL
```

Examples:

Low

Research

Medium

Generate content

High

Send email

Critical

Spend advertising budget

---

26. HUMAN-IN-THE-LOOP

The AI must NOT blindly execute high-impact actions.

Require approval for:

```
Sending campaigns
Publishing content
Spending advertising budget
Changing campaign budget
Sending mass emails
Deleting data
Changing CRM records
```

Workflow:

```
AI Recommendation
       ↓
Risk Assessment
```

```
        ↓
Approval Required?
        ↓
YES
        ↓
Human Approval
        ↓
Execute
```

---

27. AUTONOMOUS DECISION ENGINE

For every recommendation calculate:

```
Revenue Impact
Cost
Strategic Alignment
Customer Value
Competitive Advantage
Scalability
Risk
Confidence
```

Create a decision score internally, but do not expose arbitrary scores without explaining the underlying factors.

Example:

```
{
  "recommendation": "Increase LinkedIn campaign allocation",
  "reason": "",
  "expected_pipeline_impact": 250000,
  "estimated_cost": 50000,
  "confidence": 0.81,
  "risks": [],
  "approval_required": true
}
```

---

28. EXPERIMENTATION ENGINE

Create an AI experimentation framework.

Support:

```
A/B Testing
Multivariate Testing
Audience Testing
Messaging Testing
Landing Page Testing
CTA Testing
Email Subject Testing
Ad Creative Testing
```

Track:

```
Hypothesis
Experiment
Control
Variant
Metric
Result
Confidence
Decision
```

After every experiment:

```
Winner/Loser is not assumed automatically.

Evaluate statistical evidence and business impact.
```

Store successful experiments in organizational memory.

---

29. MARKETING FUNNEL ENGINE

Track:

```
Visitor
 ↓
Lead
 ↓
MQL
 ↓
SQL
```

```
 ↓
Opportunity
 ↓
Customer
 ↓
Retained Customer
 ↓
Expansion
```

Calculate conversion between every stage.

---

30. REVENUE ATTRIBUTION

Support:

```
First Touch
Last Touch
Multi-Touch
Position Based
Time Decay
```

Track:

```
Marketing Source
Campaign
Lead
Opportunity
Revenue
```

Build:

```
Campaign → Lead → Opportunity → Revenue
```

---

31. DASHBOARD

Build an executive dashboard.

Dashboard Sections

```
Revenue Overview
Lead Generation
Pipeline
Campaign Performance
Channel Performance
Content Performance
SEO
Social Media
Email
Customer Retention
Marketing ROI
AI Recommendations
```

Example:

```
Pipeline Created
$1.2M

MQL
423

SQL
127

Conversion
30%

CAC
$420

Marketing ROI
4.2x
```

---

32. AI EXECUTIVE COPILOT

Create a conversational interface.

Example questions:

```
"What are our biggest growth opportunities?"

"Why did lead generation decrease?"

"Which campaign generated the most pipeline?"

"Where should we allocate next month's budget?"

"Which customers are at risk?"

"What content should we publish next?"

"What are competitors doing?"

"Why is CAC increasing?"

"Generate a 30-day growth plan."

"Create a campaign for our new product."
```

The Copilot should retrieve real data before answering.

---

33. REPORT GENERATION

Generate:

Daily

```
Marketing Health Summary
Critical Alerts
AI Recommendations
```

Weekly

```
Marketing Performance Report
Lead Report
Campaign Report
Competitor Report
Growth Opportunities
```

Monthly

```
Executive Marketing Report
ROI
Pipeline
Revenue
CAC
LTV
Campaign Analysis
Strategic Recommendations
```

Quarterly

```
Quarterly Business Review
Growth Analysis
Market Changes
Competitive Analysis
Strategic Roadmap
```

---

34. ALERT SYSTEM

Generate alerts for:

```
Lead volume drop
Conversion drop
CAC increase
Campaign failure
Budget overspend
Website traffic drop
Competitor activity
Customer churn risk
High-value lead
High-intent visitor
Revenue opportunity
```

---

35. DATABASE DESIGN

Create PostgreSQL schemas/tables for:

```
organizations
users
roles
permissions

products
services
customers
personas
icps

competitors
market_reports

campaigns
campaign_channels
campaign_metrics

leads
lead_scores
lead_events

contacts
accounts
opportunities

content
content_versions
content_performance

keywords
seo_reports

social_posts
social_metrics

email_campaigns
email_events

customer_feedback
customer_health

experiments
experiment_results

agent_runs
agent_tasks
```

```
agent_messages

tool_calls
approvals

documents
document_chunks
embeddings

memory
episodic_memory

analytics_events
reports
alerts
recommendations
```

Every organization must have strict tenant isolation.

---

36. API DESIGN

Create FastAPI APIs.

Example:

```
POST /api/v1/organizations

POST /api/v1/agents/run

POST /api/v1/research/market

POST /api/v1/research/competitors

POST /api/v1/personas/generate

POST /api/v1/content/generate

POST /api/v1/seo/analyze

POST /api/v1/campaigns/create

POST /api/v1/leads/score

POST /api/v1/analytics/analyze
```

```
POST /api/v1/recommendations/generate

POST /api/v1/approvals/{id}/approve

POST /api/v1/approvals/{id}/reject

GET /api/v1/dashboard

GET /api/v1/reports

GET /api/v1/alerts
```

---

37. ASYNCHRONOUS EXECUTION

Long-running AI tasks must run asynchronously.

Use:

```
FastAPI
+
Redis
+
Celery / background workers
```

Example:

```
POST /campaign/analyze

API
 ↓
Create Job
 ↓
Queue
 ↓
Worker
 ↓
Agents
 ↓
Database
```

```
  ↓
Frontend polling/WebSocket
```

Use streaming for AI responses where appropriate.

---

38. OBSERVABILITY

Track every AI execution.

Store:

```
Agent
Task
Prompt Version
Model
Input Tokens
Output Tokens
Latency
Tool Calls
Cost
Result
Errors
User Approval
```

Create:

```
Agent Performance Dashboard
```

Metrics:

```
Success Rate
Failure Rate
Latency
Token Usage
Cost
Tool Success
Human Approval Rate
```

---

39. AI EVALUATION

Create evaluation datasets.

Evaluate:

Agent Quality

```
Correctness
Relevance
Groundedness
Completeness
Consistency
```

Marketing Quality

```
Lead Quality
Content Quality
Campaign Performance
Recommendation Accuracy
```

RAG

```
Retrieval Precision
Retrieval Recall
Faithfulness
Answer Relevance
```

---

40. GUARDRAILS

Implement:

```
Prompt Injection Protection
PII Detection
Data Leakage Prevention
Hallucination Detection
Tool Permission Checks
Rate Limits
Budget Limits
Approval Gates
```

```
Tenant Isolation
Audit Logging
```

The system must never expose data belonging to another organization.

---

41. COST OPTIMIZATION

Implement intelligent model routing.

Example:

```
Simple classification
→ cheap model

Summarization
→ medium model

Complex strategy
→ advanced model

Large batch processing
→ optimized batch model
```

Implement:

```
Caching
Prompt Compression
Context Filtering
Semantic Cache
Batch Processing
Token Budgeting
Model Routing
```

Track:

```
Cost per Agent Run
Cost per Campaign
Cost per Lead
Cost per Customer
```

---

42. SECURITY

Implement:

```
JWT Authentication
OAuth
RBAC
API Keys
Secrets Management
Encryption
Tenant Isolation
Audit Logs
Rate Limiting
Input Validation
Output Validation
```

Never store API keys in plain text.

---

43. FRONTEND PAGES

Create:

```
/login

/register

/onboarding

/dashboard

/ai-copilot

/agents

/agents/{id}

/market-research

/competitors

/personas
```

```
/content

/seo

/social

/email

/campaigns

/leads

/sales

/analytics

/experiments

/customers

/retention

/reports

/approvals

/integrations

/settings
```

---

44. AI COPILOT UX

The user should be able to type:

```
Create a 30-day lead generation strategy for my AI SaaS product.
```

The system should show:

```
Understanding Objective
        ↓
Researching Market
        ↓
Analyzing Competitors
```

```
        ↓
Building ICP
        ↓
Creating Strategy
        ↓
Creating Campaigns
        ↓
Generating Content
        ↓
Calculating KPIs
        ↓
Preparing Execution Plan
```

Then display the final strategy.

---

45. END-TO-END EXAMPLE

User:

> "Generate 500 qualified leads for our AI customer-support SaaS in India within 60 days with a ₹5 lakh marketing budget."

The system should execute:

```
1. Understand Goal

2. Market Research

3. Competitor Research

4. ICP Generation

5. Persona Generation

6. Customer Journey

7. Channel Analysis

8. Keyword Research

9. Content Strategy

10. Campaign Strategy
```

```
11. Budget Allocation

12. Landing Page Strategy

13. Lead Magnet

14. LinkedIn Strategy

15. Google Ads Strategy

16. Email Strategy

17. Lead Scoring

18. CRM Integration

19. Sales Enablement

20. Analytics Setup

21. Launch Plan

22. Human Approval

23. Campaign Execution

24. Monitor Results

25. Detect Underperformance

26. Optimize

27. Generate Weekly Report
```

---

46. AUTONOMOUS OPTIMIZATION LOOP

Implement this continuous loop:

```
OBSERVE
   ↓
ANALYZE
   ↓
IDENTIFY PROBLEM
   ↓
```

```
GENERATE OPTIONS
   ↓
PREDICT IMPACT
   ↓
SELECT ACTION
   ↓
APPROVAL
   ↓
EXECUTE
   ↓
MEASURE
   ↓
LEARN
   ↓
UPDATE MEMORY
   ↓
REPEAT
```

This is the core of the Agentic AI system.

---

47. AGENT COMMUNICATION

Agents must communicate through structured messages.

Example:

```
{
  "from_agent": "market_research_agent",
  "to_agent": "campaign_agent",
  "task": "campaign_planning",
  "findings": [],
  "confidence": 0.84,
  "sources": [],
  "recommendations": []
}
```

Do not pass uncontrolled natural-language context between agents when structured data can be used.

---

48. AGENT FAILURE HANDLING

If an agent fails:

```
Retry
 ↓
Alternative tool
 ↓
Alternative agent
 ↓
Fallback workflow
 ↓
Human escalation
```

Never silently fabricate missing information.

---

49. SOURCE GROUNDING

Every external factual claim should maintain:

```
Source
URL
Timestamp
Source Type
Confidence
```

Example:

```
{
  "claim": "Competitor launched feature X",
  "source": "official website",
  "url": "",
  "timestamp": "",
  "confidence": 0.95
}
```

---

50. OUTPUT FORMAT

Every strategic recommendation must follow:

```
EXECUTIVE SUMMARY
```

```
OBJECTIVE

CURRENT SITUATION

KEY INSIGHTS

RECOMMENDATION

BUSINESS IMPACT

EXPECTED REVENUE IMPACT

EXPECTED COST

EXPECTED ROI

PRIORITY

REQUIRED RESOURCES

TIMELINE

RISKS

ASSUMPTIONS

SUCCESS METRICS

RECOMMENDED ACTIONS

APPROVAL REQUIRED
```

---

51. IMPORTANT AGENT BEHAVIOR

The AI must:

- Be proactive
- Be data-driven
- Be revenue-focused
- Challenge assumptions
- Avoid generic advice
- Quantify impact when evidence allows
- Clearly identify assumptions
- Cite external evidence

- Use tools for external data
- Use RAG for organizational knowledge
- Maintain memory
- Learn from historical performance
- Ask for missing critical information
- Escalate risky actions
- Never fabricate metrics
- Never fabricate market research
- Never claim an action was executed unless the tool confirms execution

---

52. DEVELOPMENT METHODOLOGY

Do NOT attempt to build the entire platform in one step.

Build incrementally.

PHASE 1 — Foundation

Build:

```
Repository
Docker
FastAPI
Next.js
PostgreSQL
Redis
Authentication
RBAC
Multi-tenancy
```

PHASE 2 — AI Foundation

Build:

```
LLM abstraction
Prompt management
Structured outputs
Tool calling
Agent framework
LangGraph
```

```
Agent state
Agent execution logs
```

PHASE 3 — RAG

Build:

```
Document upload
Parsing
Chunking
Embeddings
pgvector
Hybrid search
Metadata filtering
Reranking
Citation tracking
```

PHASE 4 — Core Agents

Implement:

```
Market Research
Competitive Intelligence
Customer Persona
Brand Strategy
Content Strategy
```

PHASE 5 — Growth Agents

Implement:

```
SEO
Social Media
Email
Campaign
Lead Generation
Sales Enablement
```

PHASE 6 — Analytics

Implement:

```
Analytics ingestion
KPI engine
Marketing dashboard
Attribution
ROI
Reports
Alerts
```

PHASE 7 — Autonomous Agent

Implement:

```
Orchestrator
Planner
Memory
Decision Engine
Approval Engine
Optimization Loop
```

PHASE 8 — Integrations

Implement connectors for:

```
CRM
Google Analytics
Google Ads
LinkedIn
Meta
Email
Slack
Calendar
Website
```

Use OAuth/API integrations where available.

PHASE 9 — Evaluation & Security

Implement:

```
Guardrails
Evaluation datasets
Agent testing
Prompt injection protection
Audit logs
Security testing
Performance testing
Cost monitoring
```

PHASE 10 — Production

Implement:

```
Docker
CI/CD
Monitoring
Logging
Error tracking
Database backups
Scaling
Rate limiting
Production deployment
```

---

53. CODING RULES

Write production-quality code.

Use:

```
Python 3.12+
Type hints
Pydantic
Async programming
Clean Architecture
```

```
SOLID principles
Dependency Injection
Repository Pattern
Service Layer
Structured logging
Unit tests
Integration tests
```

Avoid:

```
Hardcoded API keys
Huge files
Circular dependencies
Global mutable state
Duplicated logic
Unvalidated LLM output
Uncontrolled agent loops
```

---

54. PROJECT STRUCTURE

Use a structure similar to:

```
agentic-marketing-platform/

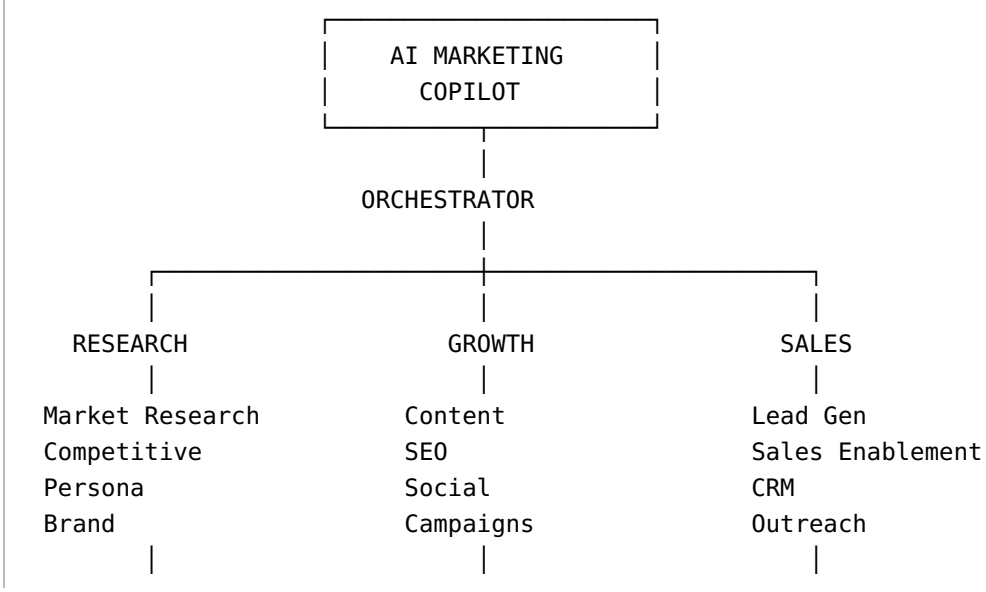
├── backend/
│
│   ├── app/
│   │   ├── api/
│   │   ├── agents/
│   │   ├── workflows/
│   │   ├── services/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── repositories/
│   │   ├── tools/
│   │   ├── integrations/
│   │   ├── rag/
│   │   ├── memory/
│   │   ├── analytics/
│   │   ├── security/
│   │   ├── evaluation/
│   │   └── core/
│   │
```

```
│   ├── tests/
│   └── requirements.txt
│
├── frontend/
│   ├── app/
│   ├── components/
│   ├── hooks/
│   ├── services/
│   ├── lib/
│   └── types/
│
├── infrastructure/
│   ├── docker/
│   ├── nginx/
│   └── deployment/
│
├── docs/
│
├── scripts/
│
├── docker-compose.yml
│
└── README.md
```

---

55. FINAL PRODUCT REQUIREMENT

The final platform should feel like:



```
        └──────────────────────┼──────────────────────┘
                               │
                          ANALYTICS
                               │
                        OPTIMIZATION
                               │
                            MEMORY
                               │
                        LEARNING LOOP
```

The platform must ultimately transform:

```
Business Goal
      ↓
AI Strategy
      ↓
AI Workforce
      ↓
Marketing Execution
      ↓
Data
      ↓
Analytics
      ↓
Optimization
      ↓
Revenue
```

Build the system as a **real production-grade Agentic AI SaaS platform**, not as a collection of demo agents.

Start with the architecture and Phase 1 implementation.

For every development phase:

1. Explain the architecture.
2. Create the required files.
3. Implement the code.
4. Explain important decisions.
5. Run tests.
6. Fix errors.
7. Show API examples.
8. Show database schema changes.
9. Show how agents interact.
10. Do not proceed to the next phase until the current phase is functional.

At every stage prioritize:

Reliability → Security → Grounded AI → Observability → Cost Efficiency → Scalability → Autonomous Optimization.